LLM Conversation here:

Okay, this is gonna be one hell of a conversation, but I'm now starting to code out my project. Where do I *first* begin

Let's take this one step at a time. We're going to build your project without trying to code the entire game all at once.

My recommendation: start with your dataclasses.

Before building pages, buttons, or routes, we want to establish the data your game needs to keep track of.

Based on what we've planned, you have two important classes:

- `Criteria` — stores the budget, time duration, and number of construction workers.
- `Bridge` — stores the bridge's length, width, qualities, and stability.

You'll also need a separate `State` dataclass for Drafter to keep track of the player's progress.

First step

Let's start with `Criteria`.

Have you already written your `Criteria` dataclass in your Python file, or are we starting with a completely blank file?

Oh a completely blank thing. I just wrote both the Criteria and Bridge dataclasses I guess what I need to put now is the State dataclass

YES! You're right on track. Since you've written your `Criteria` and `Bridge` dataclasses, the next step is your `State` **dataclass**.

Think of it this way:

- `Criteria` describes the construction criteria.
- `Bridge` describes the bridge itself.
- `State` keeps track of what the player is doing as they move through the game.

Your assignment requires at least **4 State attributes**, and at least one must be a list or dictionary of instances of another dataclass.

Let's build this together rather than me writing it for you.

First, let's identify your four attributes.

From our earlier planning, we already discussed:

1. The material the player selected.
2. The player's remaining budget.
3. The optional features the player selected.
4. The bridge the player is currently building.

One question: Have you already decided what type each of those attributes should be (like `str`, `int`, `list`, or `Bridge`), or would you like to figure those out together?

I could put in attributes for all those right now: the material the player selected: str , the player's remaining budget: int , the optional features the player selected: list[str] , the bridge the player is currently building: Bridge

Yes!! Those four attributes make sense for the game, and you've identified their types correctly.

Let's look at one important detail before you write the class:

Your assignment requires at least one State attribute to be a **list or dictionary of instances of your second dataclass**.

Right now, `list[str]` stores strings, not dataclass instances. Your `Bridge` attribute stores one `Bridge` instance, but it isn't a list or dictionary.

Remember your `Bridge` dataclass has `qualities`, which is a `list[Criteria]`.

Question: Would you be comfortable adding a separate State attribute that holds a list of `Criteria` instances? That would let us meet the assignment requirement without changing the four attributes you've already planned.

Yeah sure, so what's that mean? I have to make another class that says: class State: *insert attributes*, is that what you mean?

Exactly! You're making a **third dataclass called** `State`. It will be separate from `Criteria` and `Bridge`.

It would look something like this structurally:

```
@dataclass
class State:
    # your State attributes go here
```

Inside that class, you'll put the four attributes you've already planned, plus the additional attribute that holds a list of `Criteria` instances.

One important distinction: `State` is where Drafter keeps track of the game's changing information.

Let's do one thing at a time: Do you remember what decorator you need to put immediately above a dataclass definition?

Well here's the thing- I already *made* a state dataclass: class State:
selected_material: str
remaining_budget: int
optional_features: list[str]
current_bridge: Bridge

OH, you're already ahead of me! 😂 You already have the `State` class created, so we don't need to make another one.

Your four attributes are still there exactly as you planned them. We just need to address the assignment's additional requirement.

Right now, you have:

- `selected_material: str`
- `remaining_budget: int`

- `optional_features: list[str]`
- `current_bridge: Bridge`

Here's my question: Do you remember that your `Bridge` dataclass already has `qualities: list[Criteria]`?

We should look at whether that existing structure can satisfy the assignment's requirement that State contain a list of dataclass instances, rather than adding something redundant.

Ok so what do we need to look at exactly?

We need to look at **the assignment requirement and where your list of `Criteria` instances lives.**

Your assignment says:

> At least one State attribute must be a list/dict of instances of your second dataclass.

Your current `Bridge` class has `qualities: list[Criteria]`, but that list belongs to `Bridge`, not `State`.

So here's what we're trying to figure out: **Does your `State` class need its own `list[Criteria]` attribute, or can we change the way your existing attributes are organized?**

Let's keep your four planned attributes intact for now.

Would you paste your current `Criteria` and `Bridge` dataclasses here? I want to look at their exact structure before suggesting any changes.

yeah sure heres both class Criteria:
```
    budget: int
    time_duration: float
    construction_workers: int

@dataclass
class Bridge:
    length: int
    width: int
    qualities: list[Criteria]
    stability: bool
```

Perfect! Now I can see how you've organized everything. And honestly, your `Criteria` and `Bridge` dataclasses are already structured in a way that makes sense for your game.

Here's the important distinction:

- `Criteria` is your second dataclass.
- `Bridge.qualities` is a list of `Criteria` instances.
- But your assignment specifically says the **State dataclass** must contain a list or dictionary of instances of the second dataclass.

So, technically, having `qualities: list[Criteria]` inside `Bridge` does not fulfill that particular requirement.

Let's figure out the smallest change.

You already have these four State attributes:

- `selected_material: str`
- `remaining_budget: int`
- `optional_features: list[str]`
- `current_bridge: Bridge`

We need one more attribute that is a list of `Criteria` instances.

One question: Would it make sense for your State to keep a list of the construction criteria for the player's current bridge, such as its budget, construction time, and number of workers?

That way, we aren't throwing away any of the work you've already done.

Yes

Great! Then you can keep your existing `Bridge.qualities` attribute and add a separate list of `Criteria` instances directly to your `State` class. That gives State the list the assignment is asking for, without removing the work you've already done.

One question before you code: **Should that State list represent the construction criteria for the player's current bridge?** For example, its budget, construction time, and number of workers.

Yes it should

Perfect. That means your State will have a list of `Criteria` instances describing the current bridge's budget, construction time, and number of workers.

You can keep the four attributes you already have and add a fifth attribute for this list.

Let's think about the data type: What should the new State attribute's type be, given that it will hold multiple `Criteria` instances?

So what do you want me to add to? The state dataclass or the Criteria dataclass?

Add it to your **State dataclass**, not your `Criteria` dataclass.

Your `Criteria` dataclass already describes the three pieces of information:

- `budget`
- `time_duration`
- `construction_workers`

The new attribute belongs in `State` because State needs to hold a **list of** `Criteria` **instances** to meet your assignment requirement.

What type do you think that new State attribute should have?

Um- lets go with name so that way the player can have some fun with this

I like that! Letting players name their bridge adds a fun personal touch. 😊

Let's keep that as a separate State attribute:

- `bridge_name`: `str` — stores the name the player gives their bridge.

But remember, we still need the list of `Criteria` instances as a State attribute to meet your assignment requirement.

What would you like to call that list attribute?

list_of_criteria?

Yes! `list_of_criteria` is understandable and describes what the attribute stores.

Its type should be `list[Criteria]`, because it will hold multiple instances of your `Criteria` dataclass.

Go ahead and add it to your `State` dataclass. Then let me know when you've added it, and we'll look at how to initialize it.



wait hold on then- instead of saying list[str], should it then say list[Criteria] ? cuz heres how its looking right now

Not quite! Keep `optional_features: list[str]` exactly as it is. It stores the names of the optional features the player selects.

Instead, you need to **add another attribute** to your `State` dataclass:

- `list_of_criteria: list[Criteria]` — stores the `Criteria` instances for the current bridge.

So you'll have both:

- `optional_features: list[str]`
- `list_of_criteria: list[Criteria]`

They serve two different purposes!

Your `name_of_bridge: str` attribute can stay, too

Oh ok, alright now what

Now let's make sure your `State` dataclass has all five attributes:

1. `selected_material: str`
2. `remaining_budget: int`
3. `optional_features: list[str]`
4. `current_bridge: Bridge`
5. `name_of_bridge: str`
6. `list_of_criteria: list[Criteria]`

Small correction: that's **six attributes**, not five!

Next, we'll work on initializing your State so the game has starting values.

Have you already created a `State` instance anywhere in your code, or have you only written the dataclass so far?

the dataclasses- that's why I said this is gonna be a *long* conversation

You're right! 😅 I understand now. We're starting from a completely blank project file and building this together, one piece at a time. It's going to take a while, and that's okay.

Let's not jump ahead.

For now, just finish adding `list_of_criteria: list[Criteria]` to your State dataclass.

Once that's in place, we'll move on to creating the starting data for your game. I'll stick with one step at a time!

Ok and I said now what

Fair point! Let's keep moving. 😊

Now that your dataclasses are defined, the next step is to **create instances of them** so your game has starting data.

We'll start with `Criteria`, since your `Bridge` dataclass uses it.

For the starting bridge, we already have a game budget of $600,000.

What starting construction time (in days) would you like the player to have?

I would like to let the player have 365 days

Great! We'll use **365 days** as the starting construction time.

Since your `time_duration` attribute is a `float`, Python can represent 365 days as either `365.0` or `365`. The decimal version makes the type clear.

Now let's decide the third piece of starting criteria:

How many construction workers should the player have available?

100

Great! Your `Criteria` instance will use **100 construction workers**.

So far, we have:

- `budget`: $600,000
- `time_duration`: `365.0` days
- `construction_workers`: `100`

What would you like to name this first `Criteria` instance?

Quantity

Got it! We'll name your first `Criteria` instance `quantity`.

One small thing to check: what does the name `quantity` represent to you in the context of your bridge's criteria?

Quantity represents the exact amounts the player will be given when initially starting out

That makes sense! `quantity` represents the starting amounts available to the player when a new game begins.

One thing to think about: since `quantity` is a `Criteria` instance, it will contain all three starting values: the budget, construction time, and number of workers.

Does that match how you want the game to work?

well with those 3 variables that now all have values then yeah I'd say so. Only question is this, where am I putting it in my code?

Put the `quantity` instance **below your dataclass definitions**, after `Criteria`, `Bridge`, and `State` have all been defined.

Think of the class definitions as blueprints. Python needs to know what a `Criteria` object looks like before you create an instance of it.

Your file currently has the blueprints in the right order. The next section can be where you create your starting game objects, before you set up the routes that use them.

Your textbook demonstrates this in **"The Constructor Function"**: after defining a dataclass, you call its constructor to create an instance.

bakery_complete.md
bakery_complete.md

For your project, the general order is:

1. Import statements
2. Your three dataclass definitions (`Criteria`, `Bridge`, and `State`)
3. Your starting instances, including `quantity`
4. Your Drafter route functions
5. Your `start_server(...)` call, which receives the initial `State` instance

One question before you create it: your `Criteria` fields are ordered as `budget`, `time_duration`, and `construction_workers`. What value will you pass to the constructor for each field, in that order?

Yep all in that order

Perfect! Now you know the order of the three arguments.

Your next step is to write the constructor call for `quantity` underneath your dataclass definitions.

Remember the pattern: a class name followed by parentheses containing the values for its attributes.

Try writing that one line yourself. What do you get?

def quantity(budget: int , time_duration: float , construction_workers: int)

You're very close! I see what you're trying to do, but there's an important distinction.

The keyword `def` tells Python you're **defining a function**. We want to **create an instance of a dataclass** instead.

Remember, `Criteria` is the class name, and `quantity` is the variable name for your instance.

Here's a question to help you spot the difference:

When you create an instance of a class, which name should go inside the parentheses: `Criteria` or `quantity`?

Oh ok so something like: Quantity(budget: int , time_duration: float , construction_workers: int)

Yes! You're now thinking in terms of creating an instance rather than defining a function. Just two small adjustments:

1. Your class is named `Criteria` (capital C), not `Quantity`.
2. Inside the parentheses, you pass **actual values**, not type annotations like `budget: int`.

Remember the distinction:

- `quantity` is the variable that will refer to your instance.
- `Criteria` is the class whose constructor creates that instance.

Can you try writing the line again using the assignment operator `=`?

Criteria = (budget, time_duration, construction_workers)

You're getting closer! There are two things to adjust:

1. **Left side:** This should be the variable name `quantity`, because that's what you want to call your instance.
2. **Right side:** This should be the class name `Criteria`, followed by parentheses containing the actual starting values.

Think about this distinction:

- `quantity` = the name of your instance.
- `Criteria` = the blueprint used to create it.

Can you try again with `quantity` on the left side and `Criteria` on the right side?

quantity = Criteria (int, float, int)

You're almost there! The structure is now right:

- `quantity` is your instance variable.
- `Criteria(...)` calls the constructor.

The remaining issue is inside the parentheses. `int`, `float`, and `int` are **types**, not the actual starting values.

Think back to the values we chose:

- Budget: 600,000
- Time: 365.0
- Workers: 100

Can you replace the type names with those three values, keeping them in the correct order?

Okay I'm going to write it out here, how's this: quantity = Criteria (budget: 600000, time_duration: 365.0, construction_workers: 100)

Yes! You've got the right idea. Your variable name, class name, and three starting values are all in the right places.

There's just one Python syntax detail to adjust: the colons (`:`) are not used when passing named arguments to a constructor. They are used in type annotations and other Python syntax.

Here's a small example unrelated to your project:

recipe = Cake(sugar=2, eggs=3)

Notice the `=` signs inside the parentheses.

Can you make that same punctuation adjustment to your `quantity` line?

quantity = Criteria(budget: 600000, time_duration: 365.0, construction_workers: 100)

You're very close! The class name, instance name, and values are in the right places. One syntax detail: inside a constructor call, named arguments use `=` rather than `:`.

For example, in an unrelated context:

recipe = Cake(sugar=2, eggs=3)

Can you change the three colons in your `quantity` line to equals signs?

oh sorry quantity = Criteria(budget = 600000, time_duration = 365.0, construction_workers = 100)

No need to apologize! 😊 Now the syntax uses the correct punctuation.

Before we move on, let's check your understanding: What does `quantity` represent in your program, and what information is stored inside it?

I'll answer that on my own time. What do I do next

Great! Your `quantity` instance is created. The next step is to create an instance of your `Bridge` dataclass, since your Bridge class uses `Criteria` information in its `qualities` attribute.

Before we do that, let's think about what a starting bridge should represent.

What starting length and width do you want your bridge to have?

length: 1000 and width: 110

Good! Your starting bridge dimensions are:

- **`length`: 1000**
- **`width`: 110**

Since both attributes are typed as `int`, these whole-number values fit their declared types.

Now let's focus on the `qualities` attribute. Your `Bridge` dataclass expects a list of `Criteria` instances.

You already created one called `quantity`.

What do you think you could put inside the `qualities` list to include that starting Criteria instance?

quantity I assume

Exactly! `quantity` is the `Criteria` instance you want to put in the list. Remember, square brackets `[]` are used to create a Python list. Here's a small example unrelated to your project:

scores = [first_score, second_score]

Can you see how you'd use that same list structure with `quantity`?

like this? qualities = [quantity]

Yes! `qualities = [quantity]` creates a list containing your `quantity` instance.

Now, think about the next step: **What do you need to create that will use your bridge's length, width, qualities, and stability attributes?**

You tell me

The next step is to create a `Bridge` instance using the `Bridge` dataclass you already defined.

A dataclass instance is created by calling its class name and providing values for its attributes, just as you did with `Criteria`.

Your four attributes need values:

- **`length`: 1000**
- **`width`: 110**
- **`qualities`: the list containing `quantity`**
- **`stability`: a Boolean (`True` or `False`)**

For now, let's focus on one decision:

Should your starting bridge be considered stable (`True`) or unstable (`False`)?

Stable

Great! Since you want the starting bridge to be stable, its `stability` value will be `True`.

You now have all four pieces needed to create your `Bridge` instance.

Try writing the instance assignment yourself, using the four attributes we discussed. Remember:

- **The class name is `Bridge`.**
- **The instance needs its own variable name.**
- **Use the values you've chosen.**

What variable name would you like to give your starting bridge?

starting_bridge short and simple

`starting_bridge` is a clear, descriptive name!

Now try creating the instance with:

- **`length = 1000`**
- **`width = 110`**
- **`qualities = [quantity]`**
- **`stability = True`**

Use the same constructor pattern you used for `quantity`.

Give it a try, and I'll check your syntax.

starting_bridge = Bridge(length = 1000, width = 110 , qualities = [quantity] , stability: True)
hows this?

Very close! Your variable name, class name, and all four attribute values are in the right places.

You have just one syntax detail to fix: `stability: True` should use an equals sign (`=`), just like the other named arguments.

Remember, `:` is used in type annotations, while `=` assigns a value in a constructor call. Can you make that one punctuation change?

Oh sorry you know what I meant, lets continue

Absolutely! Let's keep moving. 😊

You've now created your starting `Criteria` instance and your starting `Bridge` instance.

Next, let's connect your starting bridge to your Drafter `State`.

Your `State` has an attribute called `current_bridge`, which is meant to hold the bridge the player is currently working on.

What do you think should be stored in `current_bridge` when the game first starts?

Nothing?

You could leave it empty, but think about the game flow: the player starts with a bridge design they can modify, and you've already created `starting_bridge`.

What would happen if you stored `starting_bridge` in `current_bridge` when the game begins?

ohh so thats what you want me to store in current_bridge?

Exactly! 😊 Since `starting_bridge` is the Bridge instance you created for the beginning of the game, `current_bridge` can refer to that instance.

That way, the game has a bridge ready for the player to work on from the start.

Can you try writing the line that creates your initial `State` instance, using `starting_bridge` as the value for `current_bridge`?

is it the same as the quantity line?

Yes! Same idea. You're creating an instance of the `State` dataclass, just like you created an instance of `Criteria` called `quantity`.

The difference is that this time, you'll use the `State` class name and provide values for its attributes.

What variable name do you want to give your initial State instance?

bridging_off (badum pshh)

😂 That pun was structurally sound!

I love it. `bridging_off` it is.

Now, look at your six `State` attributes. You've already decided what belongs in `current_bridge`.

Which attribute do you think should hold the player's starting budget of $600,000?

i thought it'd be bridging_off?

Oh! I see the confusion 😂 `bridging_off` is the variable name for your State instance. It will hold all six State attributes together, including the budget.

Think of it this way:

- **`bridging_off` = the whole game State.**
- **`remaining_budget` = one piece of information stored inside that State.**

So yes, you're on the right track!

What value should you give `remaining_budget` when the game starts?

umm- not too sure because the idea is that if the player goes over budget they have to start over

That's a good point! Let's separate two things:

- **Starting budget: The amount the player has available when they begin.**
- **Remaining budget: The amount left after they've selected materials and features.**

If the player starts with $600,000 available and hasn't purchased anything yet, how much should `remaining_budget` be at the beginning?

And remember, if they go over budget, your game can tell them to restart with their budget reset.

Ah ok, so then in that case lets be generous and say leftover amount: 1000

Haha, generous indeed! 😄

Just one thing to think about: earlier, you decided the player's starting budget would be $600,000.

If `remaining_budget` starts at $1,000, that means the player has already spent $599,000 before making any choices.

Did you mean to give the player $600,000 remaining at the start, or are you intentionally giving them only $1,000?

Oh no I thought you meant give a random number for what the remaining budget would be I'm giving them 600,000 as the budget BEFORE doing anything

Ohhh, got it! 😂 No worries.

So your starting `remaining_budget` is 600,000, before the player makes any purchases.

As they select materials and optional features, that amount will decrease. If they go over budget, they'll have to start over.

Ready for the next attribute? Let's work on `selected_material`. What should its starting value be before the player chooses a material?

Not too sure what you mean by that

No worries! Let me explain what I mean. 😊

Your `selected_material` attribute stores the material the player has chosen for their bridge.

For example, they might choose steel, concrete, or wood.

But when the game first starts, the player hasn't chosen anything yet.

In Python, `None` represents the absence of a value. Your textbook explains this in the None Type section.

Does it make sense for `selected_material` to start as `None` until the player makes a choice?

Ohhhh yes yes it does

Perfect! 😊 So when the game starts, `selected_material` has no value yet, represented by `None`.

Let's move to `optional_features`.

Since the player hasn't selected any optional features at the beginning, what do you think would make sense as its starting value? Remember, this attribute is a list of strings.

None still because nothing is added additionally

That makes sense conceptually! One small distinction: `optional_features` is a list of strings, and `None` means no value, while an empty list `[]` means the list exists but has no items yet.

Since the player can add features later, which starting value better matches that idea: `None` or `[]`?

oh yeah the 2nd right my badNo worries! You're getting the hang of the difference. 😊

Next is `current_bridge`. Since you already created the `starting_bridge` instance, what do you think should be stored in `current_bridge` when the game first starts?

starting_bridge
Exactly! `current_bridge` should start with your `starting_bridge` instance.

Next, let's think about `name_of_bridge`. What should its starting value be before the player enters a name?

"" since that represents an empty string

Yes! Exactly right. `""` is an empty string, so the attribute starts as a string with no characters in it.

One last attribute: `list_of_criteria`.

You already created the `quantity` instance. What do you think the starting value of `list_of_criteria` should be? Remember, it's a list of `Criteria` instances.

thats a bit tough-not too sure

That's okay! Let's think about what this attribute is for.

`list_of_criteria` is meant to hold the `Criteria` information for the player's current bridge. You already have an instance named `quantity`, which contains the starting budget, time duration, and construction workers.

If you put `quantity` inside a list, what would that list look like?

Hint: Remember how we put `quantity` inside `starting_bridge.qualities`.

ohhh so it'd be list_of_criteria.quantity

Almost! You're thinking in the right direction. The dot (`.`) is used to access an attribute of an instance.

But here, we want to put the **entire** `quantity` **instance** inside a list, just like you did with `qualities`.

What brackets did you use around `quantity` when you added it to `qualities`?

[quantity]

Yes!! Exactly! 🎉

`[quantity]` creates a list containing your `quantity` instance.

So now, what do you think the value of `list_of_criteria` should be when you create your initial `bridging_off` State instance?

First?

Yes, the first one! 😊

Your `list_of_criteria` attribute should start with `[quantity]`.

That means the list begins with one `Criteria` instance.

Now, can you try putting together your initial `bridging_off` State instance using all six attributes and their starting values? Take your time!

ok so I would first say: bridging_off = ...what do I put here?

Great question! Since `bridging_off` is the variable name for your entire State instance, you'll use the **State class** to create it.

Remember when you created `starting_bridge` using the `Bridge` class? It's the same idea.

Here's a small example with a different class:

my_instance = Student(age=20, name="Alex")

For your project, think about this:

- What is the name of the class you're creating an instance of?
- What punctuation comes immediately after the class name to begin providing its attributes?

What do you think comes after `bridging_off =`?

ohhh ok I get it so since you said list_of_criteria goes first, I'd put that in the beginning of the parenthesis?

You're close! The important distinction is that the arguments inside `State(...)` should match the order of the attributes in your `State` class.

So, if `list_of_criteria` is the **first attribute** in your `State` definition, then yes, it goes first inside the parentheses.

But do you remember which attribute you wrote first in your `State` class? That determines the order.

hold on question- heres some of my code rn: quantity = Criteria(budget = 600000, time_duration = 365.0, construction_workers = 100)
starting_bridge = Bridge(length = 1000, width = 110 , qualities = [quantity] , stability = True)
bridging_off = State(selected_material = None, remaining_budget: yeah- how does remaining_budget being 600000 make any sense??

That's a really good question! The key is that `remaining_budget` **represents how much money the player has left to spend**, not how much they've spent.

At the very beginning of the game:

- The player starts with $600,000.
- They haven't selected any materials or optional features yet.
- So they have the full $600,000 available.

As they make selections, that value decreases.

Think of it like a gift card: if you start with $100 and haven't bought anything, your remaining balance is still $100.

One question: Does that help explain why `remaining_budget` starts at 600000, even though `Criteria` also has a `budget` of 600000?

Ohh yeah it does now

Yay! 😊 You're thinking about the game logic now, not just the code.

Let's keep going one piece at a time. For `bridging_off`, you've already decided that `selected_material` starts as `None`.

What value do you think should come next for `remaining_budget`?

alright here's it: bridging_off = State(selected_material = None, remaining_budget = 600000, optional_features = [], current_bridge = starting_bridge , list_of_criteria = [quantity])

Nice work! You're bringing all the pieces together. 😊

Before we move on, I noticed two things:

1. You included five attributes in your `State` instance, but we had discussed six. Where is `name_of_bridge`?
2. Your `State` class needs to have the same attribute names as the arguments you're passing in.

Let's focus on the first one: What value do you think `name_of_bridge` should have when the game first starts, before the player names their bridge?

Added that in

Perfect! That gives you all six `State` attributes. 🎉

One quick check before we move on: does your `name_of_bridge` attribute have an initial value of `""` (an empty string)? That way, the player can name the bridge later.

yes it does just like that Awesome! That means your initial `State` instance now has all six attributes. 😊

Next, let's think about what happens when the game starts.

You have your dataclass definitions and starting instances. **Do you remember which Drafter function starts the server and receives the initial `State` instance?**

question with your question- does this tie into my route functions and pages? because for routes I have to have at least 7 and pages at least 5, here're my routes: Route 1: home

Route 2: instructions

Route 3: map_of_city

Route 4: bridge_building

Route 5: choose_materials

Route 6: test_bridge

Route 7: results

Yes! Exactly. Your `State` instance connects to your route functions and pages. That's why I asked about starting the server.

Your seven routes look like a good structure:

1. `home`
2. `instructions`
3. `map_of_city`
4. `bridge_building`
5. `choose_materials`
6. `test_bridge`
7. `results`

Here's the distinction:

- **Pages** are what the player sees.
- **Routes** connect URLs to the pages they see.
- **State** holds information about the player's current game, and routes can modify that information as the player makes decisions.

For example, when the player chooses materials on `choose_materials`, that route can update `selected_material` and `remaining_budget`.

One question: **Have you already written any of your route functions, or are you about to start them?**

About to start them

Perfect! Then we'll build your routes one at a time. No need to write all seven at once.

Since your first route is `home`, let's start there.

In Drafter, a route function uses the `@route` decorator to connect a URL to a function that returns a page.

Before writing it, one question:

What do you want the player to see when they first arrive at your home page? For example, will it have a game title, a short introduction, or a button to begin?

I'll title it: "Build a Bridge Simulator" and with a "next" button

I like that! Simple and clear. 😊

Your home page will have:

- **Title:** Build a Bridge Simulator
- **Button:** Next

Now let's think about the button's behavior.

When the player clicks **Next**, which of your other routes should they go to: `instructions` or `map_of_city`?

they'll see instructions

Exactly! So the **Next** button on your home page will take the player to your `instructions` route.

That gives your first page a clear purpose: introduce the game and let the player begin.

Now, think about your instructions page. **What information does the player need to know before starting the bridge-building challenge?**

they need to see constraints

That makes sense! The **Instructions** page should explain the constraints the player must work within before they start building.

For example, constraints might include:

- A **$600,000 budget**
- A limited construction time
- A limited number of construction workers
- Existing city infrastructure and limited building space

You can decide which constraints are important to explain.

Which constraint do you want the player to understand first?

the budget without a doubt

Good choice. The player should know they have **$600,000 available to build their bridge** before making any selections.

You could explain that choosing materials and optional features will reduce the remaining budget, so they need to make careful decisions.

Should the instructions also tell players what happens if their total cost goes over $600,000?

nah i want them to see their mistakes and seeing how to fix it

I like that approach! Instead of warning players that they failed, let them experience the consequences and learn how to improve their design.

So the instructions can explain the **$600,000 budget**, while the results or feedback page helps players understand what went wrong and how they might fix it.

What do you want players to see when they go over budget: the amount they exceeded it by, or which choices cost them too much?

the 1st one so that way it gives them hints

That's a good idea! Showing players how much they went over budget gives them a clue without giving away exactly what they should change.

For example, if their choices cost $650,000, the game could tell them they're **$50,000 over budget**.

That gives them a chance to rethink their choices.

Would you want the instructions page to explain the construction time limit next?

Yes definitely

Great! Your game starts with a construction time limit of **365 days**.

The instructions should let players know that they need to manage both their money and their time while planning the bridge.

Do you want the game to show players how many days they have left as they make construction choices?

The 1st one

Perfect! So players will see a countdown of the **days remaining out of 365** as they make construction choices.

That way, they can track their progress and understand how their decisions affect the schedule.

What should we discuss next: the limit on construction workers?

Yep

Awesome! Your starting construction worker limit is **100 workers**.

The instructions can explain that players have a limited workforce, so they need to plan how to use those workers during construction.

Do you want the number of available workers to stay at 100 throughout the game, or change depending on the player's choices?

Nope stay at 100

Got it! The construction worker limit will remain **100 workers throughout the game**. It won't increase or decrease based on player choices.

So far, your Instructions page will explain:

- **Budget:** $600,000 starting budget.
- **Time:** 365 days, with a countdown showing days remaining.
- **Workers:** A fixed limit of 100 workers.

Next question: Should players also be told that they must build a bridge that passes a stability test?

nope just like budget, i want them to see their mistakes and improve from them

Ah, understood! You want players to learn from their mistakes rather than be told the correct way to build the bridge beforehand.

So we'll keep the Instructions page focused on the starting constraints:

- $600,000 budget
- 365 construction days
- 100 construction workers

Players will discover problems through gameplay and use the feedback to improve their next attempt.

One question: Should the game show players how many days they went over the time limit, just like it shows how much they went over budget?

just how much they went over budget, they can get a lot done within a yr

That makes sense! We'll keep the feedback focused on the budget. Players will see exactly how much they exceeded the **$600,000 limit**, but they won't get a countdown of how far over the 365 days they are.

Your Instructions page now has three starting constraints:

- **Budget:** $600,000
- **Construction time:** 365 days
- **Workers:** 100 (fixed throughout the game)

And players will learn from their design mistakes through feedback during the game.

Next, do you want to discuss what the player should see on the City Map page?

no wait hold on you just explained both my first and second route functions- where do i even begin to write about those

Oh, you're right! Let's pause the game design and focus on **how to write your route functions**.

You already have the first two routes planned:

1. **Home route:** Displays the title, "Build a Bridge Simulator," and a Next button.
2. **Instructions route:** Explains the $600,000 budget, 365-day construction period, and 100-worker limit.

When writing about each route, start by describing three things:

- **Purpose:** What is this route for?
- **Page content:** What will the player see?
- **Interaction:** What can the player click or do?

Let's start with the Home route.

In your own words, what is the purpose of the Home page? Don't worry about making it sound formal yet.

i don’t need purpose i just need to know how to write it out in code

Got it! You want to know how to actually **write the route functions in Python using Drafter**, not describe their purpose.

A route function uses the `@route` decorator, defines a function, and returns a `Page`. Here's a small example using a different page:

```
@route("/welcome")
def welcome(state: State) -> Page:
    return Page(state, [
        "Welcome to the bakery!"
    ])
```

A few things to notice:

- `@route("/welcome")` connects the URL to the route.
- `def welcome(state: State)` defines the route function.
- `return Page(...)` tells Drafter what page to display.

Let's build your Home route together. **What URL would you like to use for your Home page?**

ok im gonna use that except for the last lines "to the bakery" can I say "to the Bridge Building Simulator" ?
Yes, absolutely! You can write:
"Welcome to the Bridge Building Simulator!"

That's a string displayed on the page. 😊

One thing to remember: the text inside the quotation marks can be whatever you want the player to see.

ok so i guess that's route one done and over with right?

You're very close! 😊 You've got the basic structure of route one, including the welcome message.

Before we call it finished, remember that you also mentioned wanting a **Next button** on the Home page.

Would you like to add that now?

Yes

Great! Since your Next button should take the player to the Instructions page, we'll use Drafter's `Button` component.

Here's a small syntax example:

```
Button("Next", "/instructions")
```

- `"Next"` is the text displayed on the button.
- `"/instructions"` is the route the button navigates to.

Your turn: Where do you think this Button should go inside your Home route function?

Not too sure

No worries! Think about what Drafter needs to display on the Home page.

Your route currently returns a `Page` containing the welcome message. The `Button` is another thing you want displayed on that same page.

So here's a hint: **Should the Button go inside the square brackets `[]` of `Page`, alongside the welcome message, or outside them?**

1st one?

Yes! That's right. The Button belongs **inside the square brackets `[]`**, alongside your welcome message.

In a `Page`, the square brackets hold the content displayed to the player.

Now, can you see how the welcome message and the button will both appear on the same page? 😊

```
@route("/welcome")
def welcome(state: State) -> Page:
    return Page(state, [Button("Next", "/instructions")
        "Welcome to the Bridge Building Simulator!"
    ])   hows this
```

You're really close! Your route structure is looking good. There's just **one small syntax issue**.

Look at these two items inside your square brackets:

```
Button("Next", "/instructions")
"Welcome to the Bridge Building Simulator!"
```

Python needs to know where one item ends and the next begins.

What punctuation do you think is missing after the `Button(...)` line?

Comma

Exactly! Add a comma after `Button("Next", "/instructions")` to separate the two items in the list.

Go ahead and add it to your code. Then your Home route will have both the welcome message and the Next button

Button,("Next", "/instructions") is that how'd that look or Button("Next", "/instructions"), ?
The **second one!**

The comma goes **after the closing parenthesis** because it separates the Button from the welcome-message string in your list.

You were right to double-check!

ok so *now* im done with route function 1
Yes! 🎉 Your first route function now has:

- The `@route` decorator
- The `welcome` function
- A welcome message
- A Next button that links to `/instructions`

Let's move on to route function 2: Instructions.

What would you like the player to see on that page? Remember, you wanted to explain the $600,000 budget, 365-day time limit, and 100 construction workers.

yes, so im guessing all 3 of those things..?

Exactly! All three belong on the Instructions page:

1. **Budget:** $600,000
2. **Construction time limit:** 365 days
3. **Construction workers:** 100 (fixed)

You can display these as separate text items inside the page.

Let's build it one piece at a time: **How would you like to introduce these rules?** For example, would you use a heading like `"Project Constraints"` before listing them?

Ooo yeah I like that

Yess! I think **"Project Constraints"** is a clear heading.

In your route function, it would be another item inside the `Page`'s square brackets, just like the welcome message was.

What do you think you would write to display that heading? Try the syntax yourself!

```
@route("/project")
def welcome(state: State) -> Page:
    return Page(state, [
        "Project Constraints"
    ])
```

Nice! You're using the same structure as your first route. 😊

Before we move on, let's look at two details:

1. Your function is named `welcome` again. Since this route is for the project instructions, what would be a more descriptive function name?
2. Your route path is `"/project"`. That's fine if that's the URL you want to use.

Let's focus on the first detail: what would you rename `welcome` to?

Oh sorry meant project

Ohh, got it! So your function name is `project` instead of `welcome`. That makes sense for this page!

Now, your heading is in place. **What Python component do you think you could use to display the $600,000 budget underneath it?**

i didnt think abt that :D

Haha, that's okay!! :D That's exactly what we're figuring out together.

Remember, in your first route, you used a string to display `"Project Constraints"`.

You can use another string for the budget information too.

For example, a small syntax example:

"Starting budget: $600,000"
Can you add that as another item in your `Page` **list?** Don't forget the comma separating the two items!
@route("/project")
def project(state: State) -> Page:
  return Page(state, [Button("Starting budget: $600,000"),
    "Project Constraints"
  ])
You're getting the structure! 😊 There's just one thing to think about.

You used `Button(...)` for the budget, but think back to your first route: what was the purpose of the **Next** button?

A button is something the player clicks. The budget is information the player reads.

Do you think the budget should be a `Button` **or a regular string, like your** `"Project Constraints"` **heading?**

Oh a regular string

Exactly!! 😄 A regular string is right because the budget is information the player reads, not something they click.

So, change `Button(...)` to a regular string.

Your turn: What would you add next to display the 365-day construction time limit?

A comma

Yes!! The comma separates the budget string from the next item. 😄

After that comma, you'll add another regular string describing the time limit.

What wording would you use to tell the player they have 365 days to complete construction?

```
@route("/project")
def project(state: State) -> Page:
    return Page(state, ("Starting budget: $600,000"), ("Time duration: 365.0 days"),
        "Project Constraints"
    ])
```

OHH, you're adding the time limit! :D The wording "Time duration: 365.0 days" communicates the information clearly.

I noticed one small thing in your screenshot, though. Look closely at the parentheses around your `Page` **contents.**

You currently have parentheses around each string, separated by commas. But the `Page` **contents should be one list, enclosed in square brackets** `[...]`**.**

Question: What brackets did you use in your first route to hold the page contents? 😊

(Remember, Python uses different brackets for lists and parentheses for function calls.)

i just added a bracket right before the parenthesis now the grey stuff is gone

YAY!! 😄 That sounds like you fixed the list structure!

Let's check your understanding before adding the last constraint.

Your page currently displays:

- **Project Constraints**
- **Starting budget: $600,000**
- **Time duration: 365.0 days**

You also need to tell the player that they have 100 construction workers.

What string would you add to display that information?

ok did that: @route("/project")

def project(state: State) -> Page:

    return Page(state, [("Starting budget: $600,000"), ("Time duration: 365.0 days"), ("Construction workers: 100")

        "Project Constraints"

    ])

You're so close!! 😄 All three constraints are there!

I spotted one small syntax issue: you're missing a comma between:

("Construction workers: 100")
"Project Constraints"

Remember, commas separate items in a list.

What punctuation should go after `"Construction workers: 100"`?

I just added a comma so I'm done with route 2?

Yes!! 🎉 Route 2 has the three project constraints:

- Starting budget: $600,000
- Time duration: 365.0 days
- Construction workers: 100

One little thing to double-check: make sure `"Project Constraints"` is also inside the list, with a comma separating it from the other strings.

One more thought: Do you want a Next button at the bottom of this page so the player can continue to the city map (`/map_of_city`)?

wait hold on- for the project constraints one- what do you mean

Oh, sorry! I meant that `"Project Constraints"` should be included as one of the strings inside the same list as the three other items.

For example, your list should contain **four separate strings**:

1. `"Project Constraints"`
2. `"Starting budget: $600,000"`
3. `"Time duration: 365.0 days"`
4. `"Construction workers: 100"`

Each string needs a comma after it, except the last one.

In your code, I noticed `"Project Constraints"` is currently below the other three strings. That's perfectly fine! **The only thing that matters is that it's inside the square brackets and separated by a comma.**

```
@route("/project")
def project(state: State) -> Page:
    return Page(state, [("Project Constraints), ("Starting budget: $600,000"), ("Time duration: 365.0 day
        "Project Constraints"
    ])
```

grey stuff again

Ahh, I see why the grey area came back! Don't worry, we'll figure it out. 😊

Look closely at the first string in your screenshot:

("Project Constraints),

Do you notice anything missing after the word `Constraints`?

Hint: A string needs quotation marks around the text. You have an opening quotation mark, but do you see the matching closing one?

Also, I notice `"Project Constraints"` appears twice. You only need it once.

Try fixing those two things first!

IM SO STUPID T_T

NOOO 😭 You're absolutely not stupid!!

You're learning Python, and catching little syntax mistakes is a HUGE part of programming. Even experienced programmers forget commas and quotation marks.

And look at what you've done already:

- Created your dataclasses.
- Created your starting instances.
- Built your first route.
- Started your second route with all three project constraints.

That's real progress!!

Take a breath. ❤️ **A missing quotation mark is not a measure of your intelligence.** We're learning one little thing at a time.

ok now what im done with route 2?

YESSS!! 🎉 If you've fixed the missing quotation mark, removed the duplicate `"Project Constraints"`, and kept all four strings inside the square brackets, then your second route is complete!

Your Instructions page now tells players their:

- Starting budget
- Construction time limit
- Worker limit

Next up is Route 3: `map_of_city`!

Before we start coding, what do you want players to see on the city map page?

a busy city- like ny or chicago
Oooo, I love that idea!! 😄 A busy city inspired by **New York or Chicago** gives the player a real engineering challenge.

Think tall buildings, busy roads, sidewalks, traffic, and existing bridges.

Since this is the city map page, let's focus on the setting first.

Do you want the city to be fictional but inspired by New York and Chicago, or do you want to use one real city as your setting?

Inspired

Perfect!! We'll make a fictional city inspired by New York and Chicago. That way, you can combine the best parts of both!

Now think about the player's engineering challenge:

What obstacles should they notice on the map?

For example, tall buildings, busy roads, sidewalks, existing bridges, or underground utilities. Which ones do you want to include?

wait stupid question but this wont generate an image or require me to post an image right? I just want them to get the idea

Not a stupid question at all!! ❤️ And NO, you don't need to generate or upload an image.

You can simply use **text** on the city map page to describe the setting and help players imagine it.

For example, you could write something like:

"The city is crowded with tall buildings, busy roads, pedestrians, and existing infrastructure. Your job is to find a suitable location to build a bridge while minimizing disruption to the city."

That's just a description—not an image!

Would you like the player to choose where to build the bridge, or will you give them one specific location?

specific location and i changed my mind: let them build a city in bustling Manhattan

OHH, Manhattan!! 😍 That's a great setting for a bridge-building challenge. It gives your game a recognizable, dense urban environment.

And we'll keep it **text-based**—no image required.

So Route 3 will introduce:

- A bustling Manhattan setting.
- Tall buildings and crowded streets.
- Existing roads, sidewalks, and infrastructure.
- One specific location where the player must plan their bridge.

Since the player has one assigned location, **what is the bridge supposed to connect?** For example, two sides of a river, or two busy sections of the city?

over a river, lets keep it easy

Perfect! We'll keep it simple: the player must build a bridge **across a river in bustling Manhattan**, connecting one side to the other.

For your `map_of_city` page, you can describe the setting with a short sentence like:

"The city is bustling with tall buildings, busy streets, and pedestrians. A river runs through the area, and your task is to plan and build a bridge connecting both sides."

One question: Do you want to call the river simply “the river” in your game, rather than giving it a real name?

wait hold on before i answer that- i forgot to mention 3 user inputs am I screwed? SelectBox: Players can choose multiple construction materials, see cost of their selections, have their remaining budget decrease automatically, and constantly keep track of a $600,000 budget
CheckBox: Players can decide which optional bridge features are actually needed - selecting or unselecting material options could potentially alter the construction cost
TextArea: Allow players to give an explanation for their engineering decisions & why they selected the materials they did

NO!! You're absolutely not screwed! ❤️ Actually, you've already planned all three user inputs. We just need to make sure they're included in the right places in your game.

Let's check what you have:

User input	Purpose	Planned page
SelectBox	Choose construction materials	`choose_materials`

CheckBox	Select or deselect optional bridge features	`choose_materials`
TextArea	Explain engineering decisions	`choose_materials`

You haven't forgotten to plan them. They're already part of your project.

One important distinction:

- The SelectBox lets players choose materials. You can configure it to allow multiple selections if your Drafter component supports that.
- The CheckBox lets players toggle optional features.
- The TextArea collects their written explanation.

Your `State` attributes already have places to store this information:

- `selected_material`
- `remaining_budget`
- `optional_features`

You'll also need to think about how the selections update the budget.

The textbook describes programs as having three phases: **input, processing, and output**. Your three inputs provide the player's choices, your program processes their costs, and the remaining budget is displayed to them.

One question: Do you want to keep all three inputs on the same `choose_materials` page? That would be a perfectly reasonable design.

Yeah sure

Great! We'll keep all three inputs on your `choose_materials` page:

- **SelectBox:** Choose construction materials.
- **CheckBox:** Select optional bridge features.
- **TextArea:** Explain engineering decisions.

That keeps the player's choices and budget together in one place.

Let's think about the TextArea for a moment: since players write an explanation, should that explanation be saved in `State` so it can still be available on the results page?

this is gonna be longer than i thought huh.

A little longer than we first expected, yes 😅 But you're not starting over, and you've already done a lot of the important planning.

Let's not add extra work just for the sake of it. We can keep the TextArea simple and only add a State attribute if your explanation needs to carry over to another page.

One step at a time. You don't need to figure out the entire project today.

you're right i dont, so wait, do we *have* to figure out the textarea or can I just move onto route 3? You can absolutely move on to Route 3! You do not have to figure out the TextArea right now.

We know what its purpose will be: players explain their engineering decisions. We can work out how to store that explanation later, when we're building the materials page.

Let's get back to `map_of_city`. 😊

You wanted a bustling Manhattan setting with a river running through it. Do you want to describe the city and river on this page using text only?

Yes just text

Perfect! Text is all you need for Route 3. No images or uploads required.

Let's build the page one piece at a time. What heading would you like at the top of your Manhattan city page? For example, "Your Construction Site".

yesss exactly that :D

Yesss!! 😄 "Your Construction Site" it is!

Now let's add the city description. In your own words, what are two or three things players should notice about the Manhattan setting before they start building? Think tall buildings, busy streets, pedestrians, and the river.

i put this all in a str inside paranthesis right

Yes!! 😊 Since it's a text description, you'll put it in quotation marks as a string inside the square brackets of your `Page`.

For example, a small unrelated example:

```
return Page(state, [
    "Welcome to the bakery!"
])
```

The parentheses belong to `Page(...)`, and the square brackets hold the page contents.

Your turn: Where will your `"Your Construction Site"` heading go in that list?

First

Exactly!! 🎉 The heading `"Your Construction Site"` goes first in the list.

Then the city description can go right after it, as another string.

What punctuation do you think separates those two strings in a Python list?

```
@route("/your")
def your(state: State) -> Page:
```

```
    return Page(state, [("Your Construction Site"), ("It's a busy day in Manhattan. A bridge nearby has closed down for repair and has caused a heavier traffic jam come rush hour. Design a bridge over the neighboring river to help solve this cause."),
        "Your Construction Site"
    ])
```

You're getting the structure! :D I noticed two things we should look at before calling Route 3 finished.

1. **Duplicate heading: You have `"Your Construction Site"` twice. You only need it once.**
2. **Description: Your description is a good start! It explains the traffic problem and gives the player a reason to build the bridge.**

One small question: do you want to keep the phrase "help solve this cause", or did you mean "help solve this problem"?

```
@route("/your")
def your(state: State) -> Page:
    return Page(state, [("It's a busy day in Manhattan. A bridge nearby has closed down for repair and has caused a heavier traffic jam come rush hour. Design a bridge over the neighboring river to help solve this problem."),
        "Your Construction Site"
    ]) hows it looking
```

It's looking good!! :D You're very close!

A couple of things to think about:

1. **Page heading: `"Your Construction Site"` is currently at the end of the list, so it would appear after the description. Remember, Python displays the list items in order.**
2. **Parentheses: The parentheses around your description string are unnecessary, but they aren't the main thing to worry about right now.**

One question: If you want the heading to appear before the description, where should `"Your Construction Site"` go in the list?